

Do We Still Need to Be Agile? What Two Cohorts of Student Projects Reveal About Planning Discipline in the GenAI-Era

Abstract—V9 abstract content will be written after the approved Methodology, Results, Discussion, Threats to Validity, and Conclusion sections.

I. INTRODUCTION

V9 introduction content will be revised after the results catalog and approved section-level evidence are available.

II. BACKGROUND AND RELATED WORK

V9 background and related-work content will be aligned with the approved empirical definitions during the editorial phase.

III. METHODOLOGY

To investigate how planning discipline and coordination evolve in AI-assisted software engineering projects, we designed an empirical study combining longitudinal repository mining, standardized instructor evaluations, multi-checkpoint student surveys, and qualitative meeting transcripts across two academic offerings.

A. Context, Course Design, and “Rules of the Game”

The study was conducted within a project-based undergraduate software engineering course at the State University of Ceará (UECE), spanning two semesters: 2025.2 ($N = 9$ analyzable teams) and 2026.1 ($N = 5$ teams), yielding a total inferential dataset of $N = 14$ team-semesters (≈ 70 participating students).

Students formed teams of approximately five members through spontaneous self-allocation, with remaining students assigned to vacant slots to ensure balanced team sizes. Teams enjoyed complete autonomy regarding their internal organization, process models, and project management methodologies (e.g., choosing freely between Agile variants, Scrum, Kanban, or unguided approaches). However, course guidelines enforced two structural constraints:

- 1) Teams were required to develop an operational software project featuring an integrated Generative AI component as a core functional requirement. Specifically, projects were expected to implement advanced architectural patterns such as Retrieval-Augmented Generation (RAG)—a technique that dynamically grounds Large Language Models (LLMs) in external knowledge bases to mitigate hallucination [1]—or LLM-driven data analytics pipelines.

- 2) Students were strongly encouraged to utilize generative AI assistants throughout all phases of software development, from requirements specification to implementation and testing.

The project lifecycle was anchored by three synchronized temporal checkpoints (T_1 , T_2 , and T_3) paced identically across both cohorts:

- T_1 (**Baseline**): Administered on the first day of class prior to any formal instruction on generative AI processes, capturing baseline perceptions, initial project scope, and early planning commits.
- T_2 (**Mid-Semester**): Conducted following structured instructional modules covering generative AI mechanics, RAG architectures, and software process models, accompanied by the second wave of survey instruments and status presentations.
- T_3 (**Final Integration**): Positioned one week prior to the final project delivery and presentation, capturing the peak pressure of system integration and deadline convergence.

B. Project Scope and Real-World Complexity

To ensure ecological validity and induce realistic coordination friction, the course guidelines explicitly prohibited trivial algorithmic exercises or “toy” applications. As part of the “rules of the game,” teams were mandated to identify and solve real-world community, educational, or business problems. This constraint forced students to grapple with high domain complexity, authentic user requirements, and the nontrivial challenge of embedding Generative AI into functional architectures.

The resulting software systems spanned diverse domains, ranging from healthcare and financial management to academic research and ecotourism. Table I details the 14 projects analyzed in this study. The breadth and complexity of these applications validate that the observed late-stage integration challenges and chronic coordination friction were driven by authentic software engineering workloads rather than artificially simplistic academic tasks.

C. Data Sources and Collection Framework

Our data collection framework triangulates four distinct evidentiary channels to capture both technical artifacts and human behavioral dynamics.

1. **Git Repositories**: Complete historical commit logs, branch structures, line change counts, and file-level provenance

TABLE I
ANALYZED STUDENT PROJECTS BY COHORT, DEMONSTRATING REAL-WORLD PROBLEM SCOPE AND GENERATIVE AI INTEGRATION

Semester	Project Name	System Description & AI Integration Scope
2026.1	FoodGuard	Web platform utilizing nutritional data and medical history to identify food allergens and prevent adverse reactions.
2026.1	RevisAI	Educational system that generates intelligent flashcards and organizes customized weekly study review cycles.
2026.1	SmartFlowIA	SaaS WhatsApp chatbot focusing on intelligent onboarding, orientation, and welcoming for university freshmen.
2026.1	Interview Prep IA	AI-driven system that analyzes resumes against job vacancies, simulates interviews, and provides personalized candidate feedback.
2026.1	EnemBot	Domain-specific chatbot designed to tutor and assist students with questions regarding the Brazilian National High School Exam.
2025.2	UECEEncontra	Intelligent virtual assistant and campus navigation system designed to help students and professors locate university resources.
2025.2	EstudaAI	Study management platform that processes multimodal files (audio, image, PDF) to generate lecture summaries, reminders, and track grades.
2025.2	CareerPath IA	Resume analysis tool that uses GenAI to match user profiles with market demands and suggest targeted skill development.
2025.2	Buliçoso	Healthcare application that utilizes LLMs to simplify complex medical leaflets and manage medication reminders for the elderly.
2025.2	EcoRoteiro	Ecotourism platform that generates personalized travel itineraries based on user profiles, physical conditions, and specific interests.
2025.2	Research Flow	Academic research assistant that searches articles, generates intelligent summaries, and formats texts to academic standards (e.g., ABNT, APA).
2025.2	ExamForge	Assessment simulator that ingests past exams and course materials to generate new tests matching specific professor grading patterns.
2025.2	SoFiA	Financial management web platform utilizing AI to help university students dynamically organize their personal finances.
2025.2	Synapse	Requirements engineering tool that summarizes technical client meetings to automatically generate functional requirements, user stories, and backlogs.

were extracted from all team repositories across the entire semester lifecycle. To mitigate the threat of unobserved out-of-band collaboration, students were explicitly instructed to centralize all project artifacts—including requirements, architectural specifications, documentation, and source code—directly within their Git repositories. This directive ensured the repository served as a comprehensive, single source of truth for the team’s entire production effort. The analysis used read-only repository histories and standardized event tables; Git remains repository evidence, not AI-use telemetry.

2. Independent Evaluator Scorecards: At each checkpoint (T_1 , T_2 , T_3), teams delivered formal status presentations. These were assessed by a panel of independent supervisors (4 evaluators in the 2025.2 cohort, and 3 in the 2026.1 cohort). To prevent groupthink and ensure scoring validity, an independent blind-scoring protocol was enforced: following the presentation and open feedback, each supervisor submitted their scores via a private electronic form. Evaluators remained blind to their peers’ scores throughout the semester. The final project score was calculated as an unweighted average of these independent assessments. Projects were assessed across four structured criteria on explicit bounded scales: *Project Evolution* (0 to 4.0 points), *Scope and Applicability* (0 to 2.0 points), *Technical Complexity* (0 to 2.0 points), and *Engagement and Participation* (0 to 2.0 points). These observations were organized by team-semester and checkpoint for analysis.

3. Multi-Checkpoint Student Surveys: Students completed identical survey instruments at T_1 , T_2 , and T_3 (totaling six instrument cycles). These surveys systematically captured

data across five core constructs: Demographics, Professional Experience, GenAI Adoption, Perceptions of AI Impact on SE Roles, and Project Expectations. To ensure replicability, the core dimensions and sample variables tracked across these checkpoints are detailed in Table II. Survey responses are cohort-by-semester/checkpoint data and carry no team identifier; M1 and M2 therefore remain at that grain.

4. Qualitative Meeting Transcripts: To capture coordination dynamics and evaluative feedback, audio recordings were collected from the three formal status presentation checkpoints. This dataset comprises the full audio records of the student presentations alongside the subsequent deliberative discussions among the supervisors in a private chat group. Crucially, these supervisory discussions were free-form qualitative reflections, completely decoupled from the numeric scorecasting process. To make the transcription data computationally feasible for analysis, the raw text was segmented into analytical chunks, yielding 100 distinct fragments for the 2025.2 cohort (35 at T_1 , 37 at T_2 , and 28 at T_3). M5 reconstructs one global corpus per checkpoint; chunks and source sessions are provenance units, not independent observations. Transcript coverage for 2026.1 is unavailable and is never imputed.

Data Anonymization and Privacy: To ensure strict ethical compliance and protect participant confidentiality, all raw data streams were subjected to a rigorous anonymization pipeline, combining deterministic heuristics with LLM-assisted redaction to scrub unstructured text across commit messages, survey free-text responses, and audio transcripts. Finalized analyti-

TABLE II
STRUCTURE OF THE MULTI-CHECKPOINT SURVEY INSTRUMENT

Construct	Evaluated Variables & Inquiries	Data Type
Demographics & Academic Background	Age, Gender, Current academic semester, Previous degrees, High school educational background.	Categorical
Professional SE Experience	Years of software development experience, Current employment status, Preferred engineering role (e.g., Backend, QA, ML), Largest past team size.	Categorical
Generative AI Adoption & Tooling	Frequency of AI usage, Specific tools adopted (e.g., ChatGPT, Gemini, GitHub Copilot, Claude), Primary use cases (e.g., Code generation, Debugging, Documentation).	Categorical & Multiple Choice
Perceived AI Impact on SE Roles	Perceived risk of extinction or severe disruption for specific roles: Backend, Frontend, QA, Project Manager, Product Manager, Scrum Master.	5-point Likert
Expectations & Motivation	Confidence levels prior to project initiation, Motivation to learn new techniques, Expected 5-year impact of AI on the industry.	5-point Likert & Open-ended

cal artifacts exclude direct personal identifiers. Detailed call records and raw payload provenance remain restricted to the internal reproducibility record.

D. Advanced Metrics Operationalization

Sections IIIA–D established what was collected. This subsection formalizes how repository history, evaluator scorecards, survey responses, and transcript text become nine metric families used for RQ1–RQ3. Each family retains its analytical grain and exposes components only when they represent distinct constructs; no component is silently combined into a new score.

A note on $T_1/T_2/T_3$ for repository-derived measures. The checkpoint labels for commit and file history denote windows bounded by each team’s evaluator presentation dates, not single calendar days. This distinction is retained throughout the repository-derived families and is made explicit in the corresponding artifacts and provenance comments.

1) *RQ1: Generative-AI adoption and role-perception metrics:* RQ1 is operationalized at cohort-by-semester/checkpoint grain because the anonymized survey responses contain no team identifier. M1 separates the perception panel from self-reported usage frequency, tasks, tools, prior experience, autonomy/tool balance, and exploratory career topics. M2 preserves role-specific Likert distributions and paired sensitivity summaries. These families describe self-report and perceived role disruption; neither is AI-use telemetry or objective extinction risk.

M1 – Perception and self-reported usage panel. The six perception families remain separate. Descriptive distributions are reported alongside raw usage fields and missingness; no heterogeneous family average is interpreted as a behavioral dependency measure. Career-topic coding remains exploratory.

M2 – Perceived role disruption. Role-specific Likert responses are summarized by cohort, checkpoint, and role. Where privacy-preserving respondent identifiers permit it, paired T1–T3 sensitivity summaries are reported without exposing identities.

2) *RQ2: Repository temporal density vs. qualitative coordination friction:* **M3 – Author activity dynamics.** M3 separates phase activity, author concentration, and rolling

seven-day activity. A secondary pooled relative-window view removes the semester dimension only for visualization; it does not replace semester-stratified observations.

M4 – Clean change dynamics. M4 separates clean-change magnitude, touching-commit intensity, artifact composition, and rolling clean churn. The consistent path-classification rule defines the included source and test paths; commit counts remain contextual rather than the primary M4 outcome.

M5 – Coordination evidence. M5 reconstructs one global transcript corpus per checkpoint and deterministically counts versioned lexical markers for alignment, handoff, integration, blockers, and rework. It reports density, composition, corpus coverage, and an auditable chunk queue. M5 is not joined or correlated with team-semester M3/M4 observations.

3) *RQ3: Upfront planning quality, clean rework, and evaluator outcomes (primary):* **M6 – Structural and declarative planning evidence.** M6a reports T1 artifact presence and repository-visible scope. Approved M6b extraction reports separate structured categories—goals, architecture/design, task decomposition, risk/dependency, and literal evidence quotes—for nine observed team-semesters; five cases without T1 commit subjects remain `unavailable_not_measured`. M6b is exploratory and is never reduced to a composite planning-quality score.

M7 – Repository inactivity. M7 reports recent seven-day repository inactivity around evaluator checkpoints and in T3-relative rolling windows. It describes repository timing and coverage only; it does not identify planning omission and is not an independent M9 predictor.

M8 – Clean rework with baseline. M8 separates clean rework magnitude, the clean rework ratio, the late-stage rolling trajectory, and baseline eligibility. A path first observed at T1 or T2 is eligible for the provenance analysis; baseline-ineligible cases remain separate and are not imputed as stability or low quality.

M9 – Stratified associations. M9 reports descriptive Spearman associations between M6a predictors and M8a/M8b or independent T3 evaluator outcomes, restricting ratio analyses to the baseline-eligible stratum. Approved M6b fields enter a separate non-composite association output for observed cases.

TABLE III
SUMMARY OF METRIC FAMILIES AND COMPONENTS

ID	Metric family and components	Primary grain
M1	Perception panel and self-reported usage components	Cohort $\times$ checkpoint
M2	Role-specific perceived disruption components	Cohort $\times$ checkpoint $\times$ role
M3	Activity, author concentration, and rolling activity	Team-semester/window
M4	Clean magnitude, intensity, composition, and trajectory	Team-semester/checkpoint/window
M5	Transcript density, composition, and coverage	Global corpus/checkpoint
M6	Structural T1 scope and structured declarative evidence	Team-semester
M7	Repository inactivity trajectory and checkpoint pattern	Team-semester/window
M8	Clean rework, ratio, trajectory, and eligibility	Team-semester/window
M9	Stratified associations and influence diagnostics	Team-semester/stratum

TABLE IV
METRIC-BOUND LANGUAGE-MODEL TASK

Task	Feeds	Structured output
Planning evidence extraction	M6b	Goals, architecture/design, task decomposition, risk/dependency, and literal evidence quotes from T_1 commit subjects.

Leave-one-out diagnostics expose influence. No M7 predictor or floor-imputation analysis is used.

Table III summarizes the nine metric families, their components, and their primary analytical grains.

E. LLM Usage in Data Extraction, Processing, and Metrics

Language-model assistance is used selectively for data extraction and is kept separate from deterministic metric construction. M1 uses coded survey responses, and M5 uses deterministic lexical coding of the transcript corpus; neither metric receives a new language-model score. M6b uses structured extraction from T_1 commit subjects for nine observed team-semesters; five team-semesters have no T_1 subjects and remain unavailable rather than being imputed.

Table IV summarizes the metric-bound language-model task and the metric component it supports.

M6b outputs were reviewed by the research team and retained for exploratory analysis as separate categories. They are not reduced to a composite planning-quality or architectural-quality score.

Persistence and auditability. Model-assisted outputs are retained with their associated prompts, responses, hashes, usage metadata, and status so that each reported value can be traced to its source evidence. These records support reproducibility while remaining outside the paper-facing narrative.

Inference boundary. All reported metric families and components are descriptive or exploratory. Missing evidence remains distinct from zero or low quality, incompatible analytical grains are not silently combined, and no causal claim is made without a design that supports it.

IV. RESULTS

This section reports the descriptive and exploratory results produced by the validated V9 metric artifacts. Results are organized by research question and retain the analytical grain of each metric; no claim combines incompatible grains or treats self-reported evidence as behavioral telemetry.

A. RQ1: Generative-AI adoption and role-perception trajectory

RQ1 is supported by two distinct evidence streams: the M1 survey panel, which summarizes six perception families and separately reports self-reported AI use, and M2, which summarizes perceived disruption agreement for six software-engineering roles. All six cohort-by-semester/checkpoint cells in M1 have complete item coverage, with 46, 41, and 40 respondents in 2025.2 and 22, 20, and 18 respondents in 2026.1 at T_1 , T_2 , and T_3 , respectively. These are repeated cohort summaries, not an identified individual longitudinal panel.

The six perception families do not move as a single trajectory. These item means use the inherited 0–4 score scale, except for the separately reported autonomy/tool-balance item, which uses its observed 1–5 response scale. Values below are reported as mean $\pm SD$. In 2025.2, mean AI benefit increases from 2.13 ± 1.00 at T_1 to 2.25 ± 0.90 at T_3 , while five-year career impact decreases from 1.48 ± 1.01 to 1.33 ± 0.83 . Autonomy/tool balance changes from 3.09 ± 0.89 to 3.23 ± 0.95 . In 2026.1, AI benefit decreases from 2.00 ± 1.07 to 1.56 ± 0.98 , whereas autonomy/tool balance increases from 2.64 ± 0.85 to 3.44 ± 0.92 , and five-year career impact changes from 1.55 ± 0.91 to 1.67 ± 0.91 . The project-expectation families remain comparatively low: at T_3 , project challenges average 0.15 ± 0.36 in 2025.2 and 0.06 ± 0.24 in 2026.1, while project feeling averages 0.35 ± 0.77 and 0.50 ± 0.86 , respectively. These values describe distinct response families and should not be interpreted as one latent measure of AI dependence.

Figure 1 presents these six families with cohort-specific standard-deviation bars. Table V reports the corresponding endpoint values in compact form.

The raw usage fields provide a separate adoption description. The share reporting daily general AI use rises from 41.3% to 45.0% in 2025.2 and from 54.5% to 61.1% in 2026.1 between T_1 and T_3 . For software projects specifically, the daily-use share changes from 23.9% to 32.5% in 2025.2 and from 40.9% to 38.9% in 2026.1. The most frequently reported tasks include general research/questions, code generation, debugging, and documentation. For example, code generation rises from 65.2% to 77.5% in 2025.2, while documentation is reported by 72.7% of the 2026.1 T_1 respondents and by

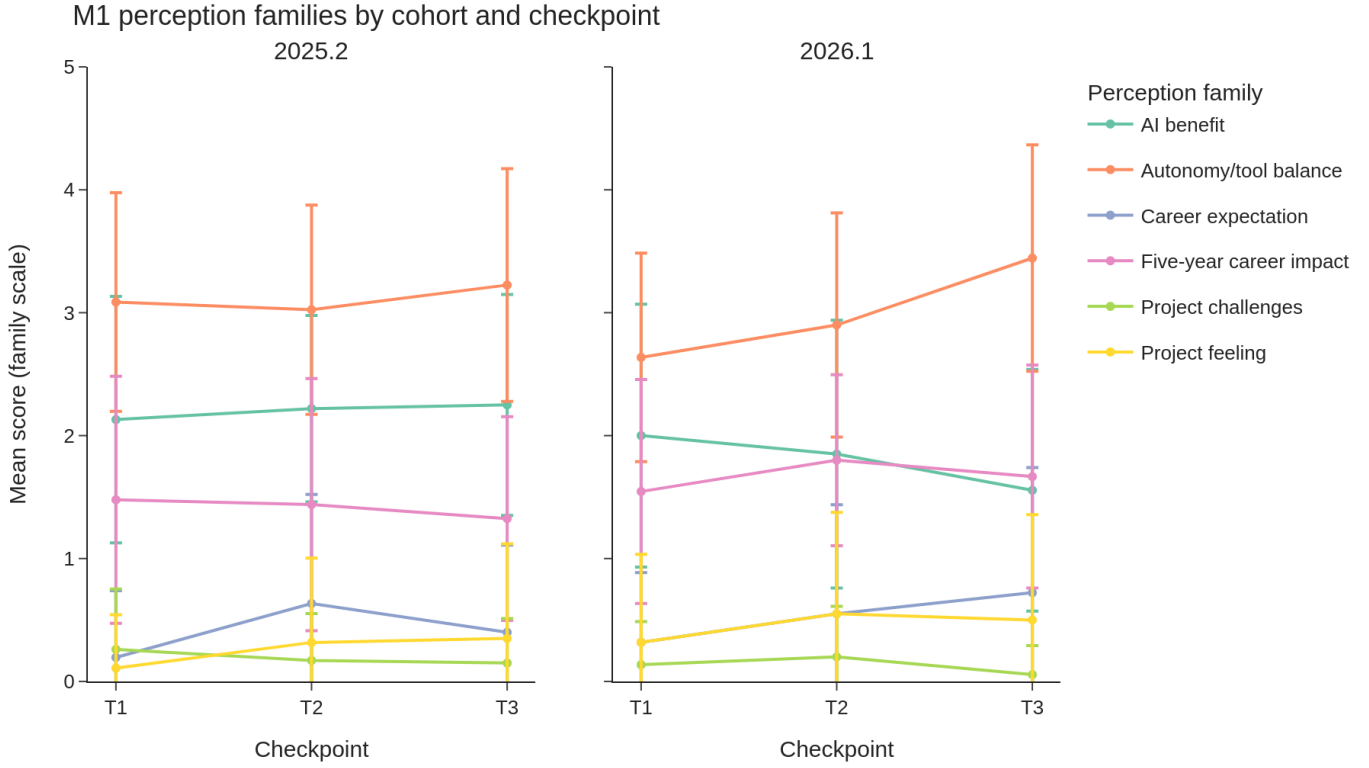


Fig. 1. M1 perception families by cohort and checkpoint. Points show means and bars show one standard deviation; family scales are retained from the underlying instruments.

TABLE V
M1 ENDPOINT SUMMARIES: MEAN $\pm$ SD

Family	2025.2 T_1	2025.2 T_3	2026.1 T_1	2026.1 T_3
AI benefit	2.13 $\pm$ 1.00	2.25 $\pm$ 0.90	2.00 $\pm$ 1.07	1.56 $\pm$ 0.98
Five-year career impact	1.48 $\pm$ 1.01	1.33 $\pm$ 0.83	1.55 $\pm$ 0.91	1.67 $\pm$ 0.91
Autonomy/tool balance	3.09 $\pm$ 0.89	3.23 $\pm$ 0.95	2.64 $\pm$ 0.85	3.44 $\pm$ 0.92
Project challenges	0.26 $\pm$ 0.49	0.15 $\pm$ 0.36	0.14 $\pm$ 0.35	0.06 $\pm$ 0.24
Project feeling	0.11 $\pm$ 0.43	0.35 $\pm$ 0.77	0.32 $\pm$ 0.72	0.50 $\pm$ 0.86

66.7% at T_3 . These are self-reported frequencies and selected activities, not instrumented measures of tool use.

Reported tool use is concentrated in a small group of widely named tools but is not constant across cohorts. ChatGPT is reported by all 2025.2 respondents at T_1 and T_3 , and by 83.3% of the 2026.1 T_3 respondents. Gemini is reported by 77.5% of 2025.2 respondents at T_3 and 77.8% of 2026.1 respondents at T_3 . Prior AI-project experience also increases across the observed checkpoints, from 41.3% to 70.0% in 2025.2 and from 31.8% to 94.4% in 2026.1. Because the respondent counts and identities are not stable individual panels, these changes cannot be interpreted as within-person learning or adoption effects.

Career-impact topic coding is retained as an exploratory supplement rather than a validated result. Its largest categories are uncertainty or conditional statements and other/insufficient evidence; for example, together they account for 56.9% of topic assignments at 2025.2/ T_1 and 65.2% at 2026.1/ T_3 .

The coding is rule-based and explicitly unvalidated, so these proportions are presented as exploratory descriptions only.

M2 shows a role-specific pattern rather than a uniform increase in perceived disruption. The values below are means on the 1–5 Likert agreement scale; each parenthetical value is the corresponding SD . In 2025.2, Frontend has the highest mean agreement at all three checkpoints: 2.74 ± 1.10 , 2.68 ± 1.08 , and 2.73 ± 1.18 . Project Manager remains among the lowest: 1.83 ± 1.02 , 1.83 ± 0.86 , and 2.08 ± 0.89 . In 2026.1, Frontend is also high, moving from 3.14 ± 1.13 to 2.65 ± 1.09 and 3.28 ± 1.36 ; Scrum Master starts at 3.14 ± 1.25 at T_1 . Agreement is therefore role-specific and cohort-dependent; it does not support a general claim that all roles become progressively more exposed to replacement or transformation. Figure 2 provides the complete role-by-cohort/checkpoint view; its color scale is fixed to the 1–5 response range.

The privacy-preserving paired sensitivity analysis reinforces

M2 perceived role disruption agreement

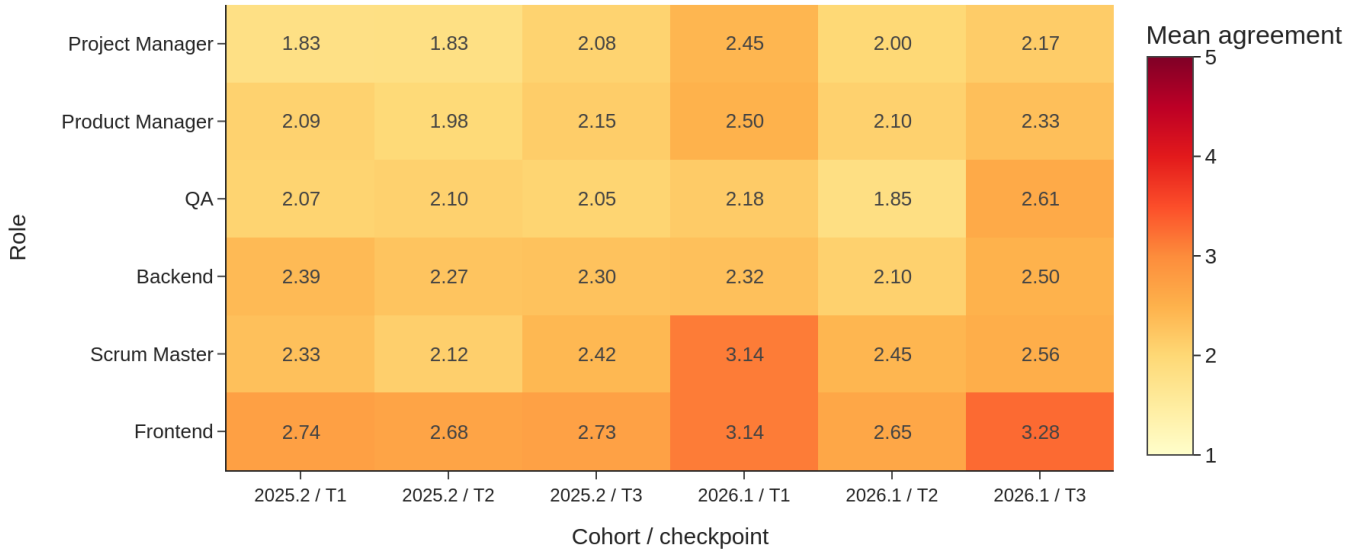


Fig. 2. M2 perceived role-disruption agreement by cohort and checkpoint. Cells show mean agreement on the 1–5 Likert scale.

this caution. There are 38 paired respondents per role in 2025.2 and 16 per role in 2026.1. Changes are mixed rather than uniformly directional: in 2025.2, Project Manager shows the largest increase (+0.32) and Backend is nearly unchanged (−0.03); in 2026.1, QA increases by +0.69, while Scrum Master decreases by −0.31. These paired values are sensitivity summaries, not causal or population-level estimates.

V. DISCUSSION

Discussion content will interpret the approved descriptive and exploratory results without causal overreach.

VI. THREATS TO VALIDITY

Threats to validity will document grain differences, Git coverage, path classification, transcript lexical coding, LLM review, sample size, and influential cases.

VII. CONCLUSION

The conclusion will be written only after the preceding editorial sections have been approved.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] A. Nguyen-Duc *et al.*, “A systematic literature review on generative ai in software engineering,” *Preprints*, May 2026. [Online]. Available: <https://www.preprints.org/manuscript/202605.1638>
- [3] IEEE Computer Society, “Transforming software engineering processes through generative ai,” *IEEE Computer*, July 2025. [Online]. Available: <https://www.computer.org/csdl/magazine/co/2025/07/11052608/27RKcQkdNg4>
- [4] Cambridge Advance Online, “The contextual gap in genai for software engineering,” July 2026. [Online]. Available: <https://advanceonline.cam.ac.uk/blog/the-contextual-gap-in-genai-for-software-engineering>
- [5] GitHub Blog, “Spec-driven development with ai: Get started with a new open source toolkit,” September 2025. [Online]. Available: <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- [6] Thoughtworks, “Spec-driven development,” 2025. [Online]. Available: <https://www.thoughtworks.com/en-br/insights/decoder/s/spec-driven-development>
- [7] Codebridge, “The hidden costs of ai-generated software: Why ‘it works’ isn’t enough,” February 2026. [Online]. Available: <https://www.codebridge.tech/articles/the-hidden-costs-of-ai-generated-software-why-it-works-isnt-enough>
- [8] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, “Manifesto for agile software development,” 2001. [Online]. Available: <https://agilemanifesto.org/>
- [9] W. Cunningham, “The wycash portfolio management system,” in *OOP-SLA ’92 Experience Report*. ACM, 1992.
- [10] M. Fowler, “Technical debt quadrant,” 2009. [Online]. Available: <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>
- [11] P. Kruchten, R. L. Nord, and I. Ozkaya, “Technical debt: From metaphor to theory and practice,” *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.
- [12] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.
- [13] T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*. Dorset House, 1987.
- [14] Z. Li, P. Avgeriou, and P. Liang, “A systematic mapping study on technical debt and its management,” *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.
- [15] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Boston, B. McGrew, S. Karthik, and *et al.*, “Evaluating large language models trained on code,” in *arXiv preprint arXiv:2107.03374*, 2021.
- [16] C. Williams, M. Jones, and R. Patel, “Generative ai and software engineering education: Opportunities, risks, and design constraints,” in *Proceedings of the International Conference on Software Engineering Education and Training*, 2024.

- [17] S. Miller, J. Ferris, and A. Chen, "Coordination signals and human-ai teaming in software engineering courses," in *Proceedings of the ACM/IEEE International Conference on Software Engineering*, 2023.
- [18] J. Pérez, A. Moreno, and E. Lauri, "Planning quality, rework, and team coordination in project-based software courses," *IEEE Transactions on Education*, vol. 65, no. 2, pp. 214–223, 2022.